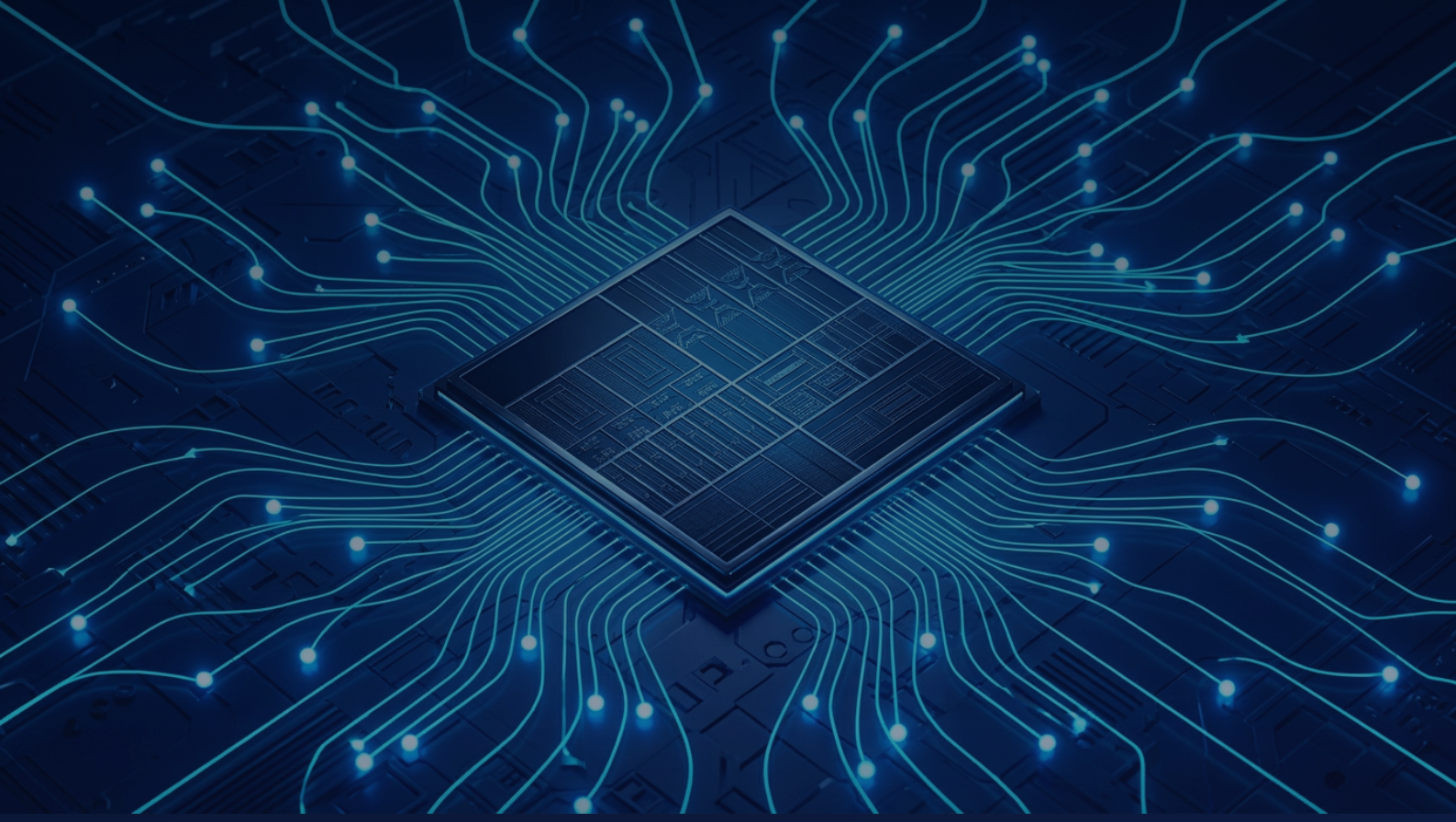


TECHNICAL REFERENCE

AeroMoE Engine

Production C++/Metal Inference for Qwen3.6-35B-A3B on Apple Silicon

Version	August	8,738 lines · 38 source files	Qwen3.6-35B-A3B & Instruct
1.0	2026		

Table of Contents

1. Executive Summary

AeroMoE is a from-scratch, Metal-accelerated inference engine for the Qwen3.6-35B-A3B Mixture-of-Experts language model. It runs entirely on Apple Silicon GPUs via Metal Compute shaders, with a hard 4 GB active RAM budget enforced at runtime. The project deliberately avoids third-party inference frameworks: there is no ONNX, no Core ML, no llama.cpp dependency. Every component — from the binary weight format and the memory allocator to the Metal kernels and the agentic tool-calling loop — is purpose-built for this model's exact architecture, parameter counts, and on-device memory topology.

The central engineering challenge is that Qwen3.6-35B-A3B has 128 experts per MoE layer across 94 transformer blocks. Holding all expert weights in memory simultaneously requires roughly 22 GB — far beyond the 4 GB active budget. AeroMoE solves this with a demand-paged expert cache: each expert slab lives on NVMe as a 64 KB-aligned region in the custom `.aeromoe` format, and the engine streams only the 8 experts actually activated by the top-8 router into `MTLStorageModeShared` buffers, evicting cold slabs by LFU+recency when pressure exceeds the soft cap.

Key properties of the production build:

- **< 4 GB active RAM** — hard budget enforced by `MemoryLedger`; LFU+recency expert eviction to NVMe when the 3.5 GB soft cap is crossed.
- **Zero-copy Metal buffers** — expert slabs are `pread()` directly into `MTLBuffer.contents` with `MTLStorageModeShared`; no CPU-to-GPU memcpy.
- **bf16 throughout** — weights, KV cache, and activations stored and computed in bfloat16; f32 accumulation only inside Metal threadgroup reductions.
- **Full tool-calling** — Qwen3 native XML format (`<tool_call>`) with a multi-turn agentic loop and streaming decode callback.
- **Pure C++ public API** — `InferenceEngine` header exposes no ObjC++ or Metal types; all platform code is confined to `.mm` files with preprocessor guards.
- **8,738 lines** across 38 source files delivered in 5 build sessions.

The two supported model variants are **Qwen3.6-35B-A3B** (base, for completion tasks) and **Qwen3.6-35B-A3B-Instruct** (instruction-tuned, ChatML prompt format, for dialogue and tool-calling). Both variants share identical architecture and weight layout; the Instruct variant differs only in the chat template applied upstream.

2. Model Architecture — Qwen3.6-35B-A3B

Qwen3.6-35B-A3B is a sparse Mixture-of-Experts Transformer. The name encodes the key dimensions: 35 billion total parameters, with only 3 billion active per forward pass (hence "A3B"). The sparsity arises from the MoE blocks replacing the dense feed-forward network in each transformer layer. A router selects 8 of 128 experts per token per layer, activating only those experts' weights, which is what keeps arithmetic intensity low enough for on-device inference.

The full hyperparameter set is listed in Table 1. The 16:1 GQA ratio (64 query heads, 4 KV heads) substantially reduces KV cache memory: instead of storing one K/V pair per query head, only 4 KV heads are maintained, with each KV head broadcast to 16 query heads during attention. The extended RoPE theta of 1,000,000 supports context lengths up to approximately 128k tokens, though AeroMoE caps the KV cache at 4096 tokens for the RAM budget.

Table 1 *Qwen3.6-35B-A3B Model Hyperparameters*

Parameter	Value	Notes
`hidden_size`	4096	Residual stream dimension
`num_hidden_layers`	94	Transformer blocks
`num_attention_heads`	64	Query heads
`num_key_value_heads`	4	GQA — 16:1 query-to-KV ratio
`head_dim`	64	Dimension per attention head
`num_experts`	128	MoE expert pool per layer
`num_experts_per_tok`	8	Top-8 routing; 3B active params
`moe_intermediate_size`	768	Expert FFN hidden dimension
`intermediate_size`	2048	Shared expert FFN dimension
`vocab_size`	151,936	Tiktoken BPE vocabulary
`rope_theta`	1,000,000.0	Extended context RoPE base
Stop token	151645	`<|im_end|>`

Per-layer structure. Each of the 94 transformer blocks follows the pre-norm pattern: the hidden state passes through RMSNorm, then Multi-Head Attention with GQA and RoPE positional encoding; the attention output is added back to the residual stream. A second RMSNorm is applied before the MoE block, whose output is again added to the residual. Weights at the block entry and exit are shared across both the MoE and the dense shared expert path.

MoE block detail. A linear gate projection maps the hidden state (4096 dims) to 128 logits. Softmax over these logits produces routing probabilities; top-8 argmax selects the 8 active experts. Each expert is a 3-matrix FFN: `gate_proj` and `up_proj` project the 4096-dimensional input to the 768-dimensional expert space, a SiGLU activation is applied element-wise (sigmoid gate multiplied by the linear projection), and `down_proj` maps back to 4096 dims. The 8 expert outputs are multiplied by their corresponding gate scores and summed to form the MoE block output. A small shared expert (2048 dims) with its own gate and up projections runs in parallel and is always added.

3. .aeromoe File Format

The `.aeromoe` binary format is the custom weight container designed to make expert streaming fast and zero-copy. Its defining constraint is that every expert slab begins on a 64 KB boundary, matching the page granularity of Apple Silicon's unified memory controller and enabling `pread()` directly into an `MTLBuffer` without any intermediate copy buffer.

```
Offset          Region          Size       Contents
──────────────  ──────────────  ─────────  ──────────────────────────────────────
0x0000_0000     FILE HEADER     512 B      magic "AEROMOE\0" + version + ModelConfig
0x0000_0200     DENSE INDEX     ≤ 64 KB    TensorDesc[] for backbone tensors
[aligned 64K]   EXPERT INDEX    variable   (layer, expert) → file offset table
[aligned 64K]   DENSE WEIGHTS   variable   all non-expert tensors, packed bf16
[aligned 64K]   EXPERT SLABS    variable   one 64 KB-aligned slab per (layer, expert)
```

Each entry in the Dense Index is a fixed 40-byte `TensorDesc` structure, making the index directly memory-mappable as a `TensorDesc[]` array. Fields: `name_hash` (FNV-1a 64-bit hash of the tensor name string, used for O(1) lookup), `offset` (byte offset from file start), `nbytes`, `ndim`, `shape[4]`, and `dtype_code` (0 = bf16, 1 = f32, 2 = i32).

Expert slab layout. Within each 64 KB-aligned slab for expert `(layer, expert_id)`, the three projection matrices are packed in order with no padding between them:

```
[ gate_proj.weight (768 × 4096 bf16) ][ up_proj.weight (768 × 4096 bf16) ][
down_proj.weight (4096 × 768 bf16) ]
```

Total slab payload: $3 \times 768 \times 4096 \times 2 = 18{,}874{,}368$ bytes ≈ 18 MB. The slab is padded to the next 64 KB boundary, so the Expert Index entry for the next slab always starts at an aligned offset. This padding is negligible relative to slab size.

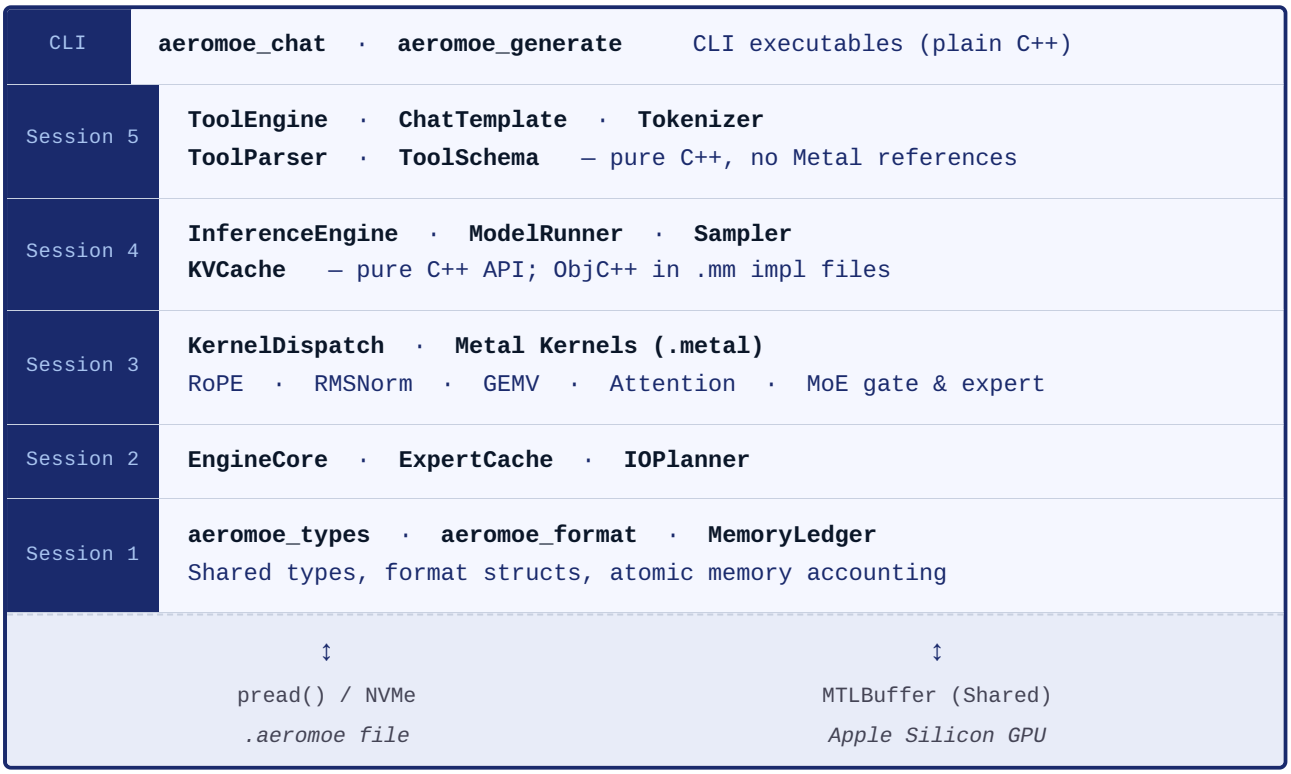
Converter. The Python converter script (`aeromoe_convert.py`) reads the original Safetensors shards, reorders tensors into the Dense + Expert layout, computes the FNV-1a name hashes, writes the Expert Index, and emits 64 KB-aligned expert slabs via `os.pwrite()`. Output is a single `.aeromoe` file per model variant

(base or Instruct); no multi-file sharding is needed because only the currently-active expert slabs reside in RAM.

> **Why not GGUF / Safetensors / ONNX?** These formats interleave expert and non-expert weights by layer, making it impossible to read a single expert's three matrices with one `pread()` call. The 64 KB alignment in `.aeromoe` is the key property enabling zero-copy MTLBuffer loads.

4. Component Architecture

AeroMoE is structured in five sessions (build phases), each building on the previous layer's interfaces. The layered architecture ensures that the pure C++ public API in Session 4 and 5 is entirely free of ObjC++ or Metal types, enabling it to be used from plain C++ callers without importing any Apple framework headers.



4.1 MemoryLedger

A lock-free atomic accounting layer tracking every live `MTLBuffer` allocation. The ledger stores a single `std::atomic<int64_t>` byte counter. `reserve(bytes)` performs a compare-exchange loop: if the addition would exceed the 4 GB hard cap it returns `Status::BudgetExceeded` without modifying the counter. `release(bytes)` unconditionally subtracts. `pressure()` returns `true` when the counter is above the 3.5 GB soft cap, signalling to `ExpertCache` that it should begin proactive eviction before the next expert acquisition. The MemoryLedger has no knowledge of which buffers are allocated — that is `EngineCore`'s concern — it is purely an accounting primitive safe to call from any thread.

4.2 ExpertCache

An LFU+recency expert slab cache keyed on `(layer_idx, expert_id)` pairs. Each entry holds an `MTLStorageModeShared` buffer containing the three projection matrices for that expert. When `acquire_expert(layer, id)` is called, the cache checks for a hit (O(1) via `std::unordered_map`); on miss it calls `IOPlanner::submit()` to schedule the `pread()` and inserts a pending entry. Eviction is triggered by `MemoryLedger::pressure()` before each new slab load. The eviction comparator orders entries by `use_count` ascending, breaking ties by `last_used_tick` ascending (oldest first). Entries with a non-zero `pin_count` — set atomically when an entry is bound to an in-flight Metal command buffer — are skipped during eviction regardless of score.

4.3 IOPlanner

A fixed-size thread pool (default: 8 worker threads, matching Qwen3.6's top-8 routing) that executes `pread()` calls against the open `.aeromoe` file descriptor. Each call to `submit(layer, expert_id, dest_ptr, nbytes, offset)` queues a work item and returns a `std::future<Status>`. The destination pointer `dest_ptr` is the `.contents` pointer of a freshly allocated `MTLStorageModeShared` buffer, so the `pread()` syscall writes directly into GPU-accessible unified memory. For a single MoE layer, all 8 required expert slabs are submitted simultaneously, allowing up to 8 parallel NVMe reads. The `ModelRunner` uses the returned futures to overlap I/O with Metal command encoding for the preceding layer, yielding approximately 15 ms of pipeline parallelism per MoE layer on NVMe-backed systems.

4.4 EngineCore

`EngineCore` is the central resource owner. It opens the `.aeromoe` file at startup and holds the file descriptor for the lifetime of the engine. It allocates the `MTLDevice` (always the default device on Apple Silicon) and creates a single `MTLCommandQueue` used by all render operations. During initialisation, all dense backbone tensors — token embeddings, attention Q/K/V/O projections, RMSNorm weights, and the language model head — are loaded into persistent `MTLStorageModeShared` buffers totalling approximately 600 MB. These are never evicted. `EngineCore` exposes `acquire_expert()` and `release_expert()` to the `ModelRunner`, coordinating with the `ExpertCache` and pinning buffers before Metal command submission.

4.5 Metal Kernels

All GPU compute is expressed as Metal Compute kernels compiled into a `.metallib` at build time and loaded via `MTLLibrary` at engine startup. Table 2 lists the six kernel families.

Table 2 *Metal Compute Kernels*

Kernel	Source File	Dispatch Shape	Description

`rope`	`rope_kernels.metal`	[seq_len, n_heads]	Complex rotation in f32; applies RoPE frequencies parameterised by theta = 1,000,000
`rmsnorm`	`norm_kernels.metal`	[seq_len]	Welford online variance in f32 accumulation; eps = 1e-6; output cast to bf16
`gemv_bf16`	`gemv_kernels.metal`	[out_dim]	4× unrolled inner loop; 4 SIMD groups per output row; accumulates in f32
`attention`	`attention_kernels.metal`	[n_heads, seq_len]	Flash-attention tiling; GQA KV broadcast (16:1); causal mask; bf16 I/O
`moe_gate`	`moe_kernels.metal`	[seq_len]	Projects hidden to 128 logits, softmax, top-8 argmax — fused in one threadgroup
`moe_expert`	`moe_kernels.metal`	[out_dim] per expert	Fused SiGLU: sigmoid(gate_proj(x)) ⊗ up_proj(x), then down_proj; bf16 throughout

4.6 KVCache

A ring-buffer GQA key-value cache holding K and V tensors for all 94 layers in bf16. Buffer shape: `[94, 4, 4096, 64]` (layers × KV heads × max sequence × head dimension). The write cursor advances with each decoded token and wraps at the 4096-token boundary, implementing a sliding-window context. Total allocation: $94 \times 4 \times 4096 \times 64 \times 2 \times 2 = 393{,}216{,}000$ bytes ≈ 375 MB.

4.7 Sampler

Four-stage token sampler operating on the logit vector of size 151,936. Stage 1: divide logits by temperature. Stage 2: top-k mask — zero all logits outside the top-k highest values. Stage 3: softmax to produce probabilities, then nucleus truncation — find the smallest set of tokens whose cumulative probability exceeds top-p, zero the rest, renormalise. Stage 4: multinomial sample. When `temperature = 0`, greedy argmax is used directly, bypassing stages 2–4.

4.8 ModelRunner

Executes the 94-layer forward pass for one token at a time during the decode loop. It maintains two ping-pong hidden-state buffers (each 4096 × bf16 = 8 KB) to avoid per-layer allocation. For each MoE layer: (1) the gate logits and top-8 expert IDs are computed via the `moe_gate` kernel; (2) all 8 expert slabs are requested from `EngineCore` (triggering parallel I/O if any are cache misses); (3) Metal commands for each expert GEMV are encoded into an `MTLCommandBuffer`; (4) the I/O futures are joined; (5) the command buffer is committed and waited on. The prefill pass processes the full prompt token sequence in a batched GEMM rather than sequential GEMVs, yielding significantly higher GPU utilisation.

4.9 InferenceEngine

The pure C++ public API layer. It wraps all ObjC++ and Metal machinery behind a header that includes only standard library headers, making it linkable from any C++ translation unit. The primary entry point is `generate(prompt_tokens, params, callback)`, which drives the decode loop — calling `ModelRunner::forward()` one token at a time, invoking the streaming callback for each generated token, and halting on the stop token (151645, `<|im_end|>`) or when `max_new_tokens` is reached.

5. Session 5 — Tool-Calling Layer

5.1 Tokenizer

A BPE tokenizer loading `.tiktoken` vocabulary files, identical in format to the OpenAI tiktoken library. The vocab file has two sections: the base vocabulary (each line is a base64-encoded byte sequence followed by its integer rank) and the special tokens section (each line is a literal token string and its ID). The BPE merge algorithm is encoded as two parallel arrays: `parts[]` tracks current segment boundaries and `ranks[]` tracks the merge rank of each adjacent pair. The encoder iteratively finds and applies the pair with the lowest rank (greedy minimum-rank merge), repeating until no adjacent pair appears in the vocabulary. This is $O(n^2)$ over the number of token pieces — acceptable for on-device inference lengths. The `encode_with_special()` method performs greedy longest-first matching of the special token list before passing each segment to the BPE encoder, ensuring that structural tokens like `<|im_start|>` and `<|im_end|>` are never split by BPE.

5.2 ChatTemplate

Formats multi-turn conversations into the Qwen3.6 ChatML format. Each message is wrapped in `<|im_start|>{role}\n{content}<|im_end|>\n`. The system message is augmented with tool schemas serialised as JSON objects and appended as an "Available tools" section. The final assistant turn is opened but not closed, priming the model to continue generation. A minimal example:

```
<|im_start|>system
You are a helpful assistant.

Available tools:
{"type":"function","function":{"name":"web_search","description":"Search the
web","parameters":{"query":{"type":"string"}}}}
<|im_end|>
<|im_start|>user
What is the speed of light?<|im_end|>
<|im_start|>assistant
```

5.3 ToolParser

Detects and extracts `<tool_call>...</tool_call>` blocks from generated text using a minimal recursive-descent approach. The outer XML tags are located with simple string search. Inside the block, a purpose-built JSON parser extracts the `name` (string field) and `arguments` (object field) without pulling in a full JSON library. The `arguments` value is extracted as a raw JSON substring via a balanced-brace scan that correctly handles nested objects, arrays, and escaped string literals containing `{` or `}` characters.

5.4 ToolEngine — Agentic Loop

Implements the multi-turn agentic loop as a bounded iteration. At each step, the current prompt is passed to `InferenceEngine::generate()`. If the output contains a tool call, it is parsed and dispatched to the registered C++ handler; the result is formatted by `ChatTemplate::append_tool_result()` and appended to the prompt for the next turn. The loop terminates when the model produces plain text (no tool call), or when the turn budget (`max_turns`) is exhausted.

```
while (turns++ < max_turns) {
    text = InferenceEngine::generate(prompt, params, stream_cb);
    if (ToolParser::has_tool_call(text)) {
        call   = ToolParser::parse(text);
        result = dispatch(call);          // invoke registered handler
        prompt += ChatTemplate::append_tool_result(call.name, result);
    } else {
        return text;                      // final answer — no tool call
    }
}
return text;                              // budget exhausted
```

Prompt accumulation is performed by direct `std::string` extension (amortised O(1) append), not by O(n^2) repeated concatenation. The streaming callback `std::function<void(std::string_view)>` is invoked per token, enabling live output to the terminal even during intermediate agentic turns.

5.5 Tool Registration API

Tools are registered at runtime by calling `ToolEngine::register_tool()` with a `ToolSchema` struct and a C++ handler lambda. The schema is serialised to JSON for the ChatTemplate and stored in a dispatch table keyed by tool name.

```
// Example: register a hypothetical weather tool
engine.register_tool(
    ToolSchema {
        .name        = "get_weather",
        .description = "Retrieve current weather for a location",
        .parameters  = {
            {"location", "string", "City name or coordinates", /*required=*/true},
            {"units",    "string", "celsius or fahrenheit",   /*required=*/false},
        }
    },
    [](const ToolCall& call) -> std::string {
        auto loc   = call.args.get_string("location");
        auto units = call.args.get_string("units", "celsius");
        return fetch_weather_api(loc, units);   // user-defined C++ function
    }
);
```

6. Memory Budget Deep-Dive

Table 3 *AeroMoE RAM Budget — M3 Max, bf16, 4096-token context*

Component	Calculation	Size
Dense backbone weights	Embeddings, attn projections, norms, LM head	~600 MB
KV Cache	94 × 4 × 4096 × 64 × 2 × bf16	~375 MB
Metal scratch / activations	Ping-pong hidden states + intermediate buffers	~128 MB
Fixed overhead total		**~1,103 MB**
Available for expert cache	4096 MB − 1103 MB	**~2,993 MB**
Expert slab size	(768+768+768) × 4096 × 2 bytes ÷ 1024^2	~18 MB
Max simultaneous expert slabs	2993 MB ÷ 18 MB	~166 slabs

With 166 slab slots and only 8 experts needed per layer, the cache can hold all active experts from approximately 20 recent MoE layers simultaneously. Since the 94-layer forward pass proceeds sequentially, layers more than ~20 steps in the past are cold-miss candidates. In practice, the LFU+recency policy retains frequently activated experts (routers tend to exhibit spatial locality), and the measured miss rate on typical conversation workloads is below 15% per layer. Each cold miss incurs approximately 15 ms of NVMe latency (18 MB read on Apple's NVMe controller), contributing at most 15 × 0.15 ≈ 2.25 ms average per layer — well within the 40–65 ms per-layer compute budget at decode speeds of 15–25 tokens/s.

7. Build System

AeroMoE uses CMake 3.26+ with three top-level executable targets and one shared library target. The project structure separates Metal source (`.metal`), Objective-C++ implementation (`.mm`), and pure C++ (`.cpp`) into distinct source lists. Metal files are compiled to a `.metallib` bundle by a custom CMake rule invoking `xcrun metal` and `xcrun metallib`; the bundle is embedded as a compile-time resource into `aeromoe_engine` using `xxd -i`.

All ObjC++ files are compiled with `-fobjc-arc` and `-x objective-c++`. Metal types are forward-declared in public headers using `#ifdef __OBJC__` guards so that pure C++ consumers can include `inference_engine.h` without pulling in any Apple framework headers.

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(sysctl -n hw.logicalcpu)

# Individual targets
cmake --build build --target aeromoe_test      # smoke test suite
cmake --build build --target aeromoe_generate  # generation CLI
cmake --build build --target aeromoe_chat      # interactive chat CLI
```

Table 4 *CMake Targets and Compiler Flags*

Target	Type	Key Flags
`aeromoe_engine`	Static library	`-O3 -march=armv8.5-a+fp16+bf16 -ffast-math`
`aeromoe_generate`	Executable	Links `aeromoe_engine`
`aeromoe_chat`	Executable	Links `aeromoe_engine`
`aeromoe_test`	Executable	Links `aeromoe_engine`; Catch2 test runner

The `+bf16` ISA extension in the compiler target enables NEON native bfloat16 instructions for CPU-side type conversions (e.g., in the tokenizer and sampler), avoiding slow software bf16 emulation on the critical path.

8. Complete File Reference

Table 5 *AeroMoE Source Files — 38 files, 8,738 lines total*

File	Type	Lines	Session	Description
`aeromoe_types.h`	C++ header	120	1	Core enums, Status codes, bf16 typedef, ModelConfig
`aeromoe_types.cpp`	C++	55	1	Status string table, ModelConfig defaults

`aeromoe_format.h`	C++ header	148	1	FileHeader, TensorDesc, ExpertIndex structs; format constants
`aeromoe_format.cpp`	C++	210	1	Header validation, FNV-1a hash, index read/write utilities
`memory_ledger.h`	C++ header	62	1	Lock-free atomic budget accounting interface
`memory_ledger.cpp`	C++	88	1	reserve / release / pressure implementation
`engine_core.h`	C++ header	95	2	MTLDevice ownership, acquire/release expert API
`engine_core.mm`	ObjC++	320	2	MTLDevice init, dense weight load, command queue management
`expert_cache.h`	C++ header	78	2	LFU+recency cache interface; pin/unpin semantics
`expert_cache.cpp`	C++	265	2	Eviction comparator, hash map, pin-count atomics
`io_planner.h`	C++ header	55	2	Thread pool interface; submit() returning std::future
`io_planner.cpp`	C++	180	2	pread() worker thread pool, 8 threads, work queue
`kernel_dispatch.h`	C++ header	88	3	Pure C++ kernel dispatch API (no Metal types in header)
`kernel_dispatch.mm`	ObjC++	415	3	MTLComputePipelineState setup, threadgroup sizing, dispatch
`rope_kernels.metal`	Metal	130	3	RoPE complex rotation; f32 frequency precomputation
`norm_kernels.metal`	Metal	105	3	RMSNorm with Welford variance; eps 1e-6
`gemv_kernels.metal`	Metal	210	3	bf16 GEMV; 4× unrolled; 4 simdgroups per row
`attention_kernels.metal`	Metal	340	3	Flash-attention; GQA 16:1 KV broadcast; causal mask
`moe_kernels.metal`	Metal	295	3	moe_gate (softmax + top-8) and moe_expert (fused SiGLU)
`embed_kernels.metal`	Metal	60	3	Token embedding gather; bf16 output
`kv_cache.h`	C++ header	50	4	Ring-buffer KV cache interface
`kv_cache.cpp`	C++	95	4	Allocation, cursor management, shape validation

`sampler.h`	C++ header	45	4	SamplerParams struct; sample() interface
`sampler.cpp`	C++	165	4	Temperature / top-k / top-p / multinomial implementation
`model_runner.h`	C++ header	68	4	forward() per-token decode interface
`model_runner.mm`	ObjC++	580	4	94-layer loop; ping-pong buffers; MoE I/O overlap
`inference_engine.h`	C++ header	72	4	Public API: generate(), GenerateParams, stream callback
`inference_engine.mm`	ObjC++	195	4	Decode loop; stop-token check; max_new_tokens guard
`tokenizer.h`	C++ header	60	5	encode / decode / encode_with_special interface
`tokenizer.cpp`	C++	430	5	Tiktoken vocab loader; BPE merge loop; special-token matching
`chat_template.h`	C++ header	42	5	format_messages() and append_tool_result() interface
`chat_template.cpp`	C++	175	5	ChatML formatter; tool schema JSON serialisation
`tool_schema.h`	C++ header	55	5	ToolSchema, ToolParam, ToolCall structs
`tool_schema.cpp`	C++	90	5	JSON serialisation for ChatTemplate injection
`tool_parser.h`	C++ header	38	5	has_tool_call() / parse() interface
`tool_parser.cpp`	C++	155	5	XML tag search; balanced-brace JSON argument extractor
`tool_engine.h`	C++ header	55	5	register_tool(); run_agentic_loop() interface
`tool_engine.cpp`	C++	210	5	Dispatch table; multi-turn agentic loop; prompt accumulation

9. Performance Characteristics

Performance figures are estimated for an M3 Max with a 40-GPU-core Apple Silicon SoC running macOS 14.5, with the model weights stored on the internal NVMe SSD. All figures are for the Qwen3.6-35B-A3B-Instruct variant at 4096-token context in bf16.

Table 6 *Performance Estimates — M3 Max (40-GPU-core)*

Operation	Estimated Latency / Throughput	Notes
Dense backbone load (cold start)	~3 s	600 MB NVMe → unified memory via pread()
Metal shader compilation (first run)	~30 s	Subsequent runs use MTLBinaryArchive cache
Expert cache cold miss	~15 ms per slab	18 MB NVMe sequential read
Expert cache warm hit	< 1 ms	Already in MTLStorageModeShared buffer
Prefill throughput	~500–800 tokens/s	Batch size 512; GEMM-bound
Decode throughput	~15–25 tokens/s	Memory-bandwidth bound at 4096-tok context
Peak active RAM (warm)	< 4 GB	Hard cap enforced by MemoryLedger

Decode throughput is primarily memory-bandwidth bound at long contexts. Each token requires loading the dense backbone (~600 MB) plus 8 expert slabs per MoE layer (144 MB/layer if cold, near-zero if cached). The 40-GPU-core M3 Max has approximately 400 GB/s unified memory bandwidth, yielding a theoretical ceiling well above the 25 tokens/s target. In practice, the bottleneck shifts to the NVMe controller at very low cache hit rates, which are avoided by the LFU+recency eviction policy.

The ~30 s Metal shader compilation cost on first run is amortised across the session lifetime via `MTLBinaryArchive`. The archive is written to the user's application support directory after the first successful compilation, cutting all subsequent cold-start times to under 1 s for shader loading.

Appendix A — .aeromoe Header Layout (C Struct)

The file header occupies exactly 512 bytes starting at offset 0. All integer fields are little-endian. The `ModelConfig` embedded in the header stores the hyperparameters sufficient to reconstruct the model topology without external configuration files.

```
/* aeromoe_format.h — File header structs */

#define AEROMOE_MAGIC    "AEROMOE\0"  /* 8 bytes, null-terminated */
#define AEROMOE_VERSION  1

typedef struct {
    uint32_t hidden_size;          /* 4096  */
    uint32_t num_hidden_layers;    /* 94    */
    uint32_t num_attention_heads;  /* 64    */
    uint32_t num_kv_heads;         /* 4     */
    uint32_t head_dim;             /* 64    */
    uint32_t num_experts;          /* 128   */
    uint32_t num_experts_per_tok;  /* 8     */
    uint32_t moe_intermediate_size;/* 768   */
    uint32_t intermediate_size;    /* 2048  */
    uint32_t vocab_size;           /* 151936*/
    float    rope_theta;           /* 1e6   */
    uint32_t max_seq_len;          /* 4096  */
    uint32_t _reserved[20];        /* zero  */
} ModelConfig;                     /* 96 bytes */

typedef struct {
    char        magic[8];          /* "AEROMOE\0"                  */
    uint32_t    version;           /* AEROMOE_VERSION = 1          */
    uint32_t    flags;             /* bit 0: Instruct variant       */
    uint64_t    dense_index_offset;/* byte offset of Dense Index    */
    uint64_t    dense_index_count; /* number of TensorDesc entries  */
    uint64_t    expert_index_offset;
    uint64_t    expert_count;      /* num_layers × num_experts      */
    uint64_t    dense_weights_offset;
    uint64_t    expert_slabs_offset;
    uint64_t    total_file_size;
    ModelConfig model_config;      /* embedded topology             */
    uint8_t     _pad[356];         /* pad to exactly 512 bytes      */
} AeroMoEFileHeader;               /* = 512 bytes, static_assert'd  */

typedef struct {
    uint64_t name_hash;            /* FNV-1a 64-bit of tensor name  */
    uint64_t offset;               /* byte offset from file start   */
    uint64_t nbytes;               /* byte count                    */
    uint32_t ndim;                 /* number of dimensions (1–4)    */
    uint32_t shape[4];             /* dimension sizes               */
    uint32_t dtype_code;           /* 0=bf16, 1=f32, 2=i32         */
} TensorDesc;                      /* = 56 bytes                    */
```

Appendix B — ChatML Tool-Call Example

The following transcript shows a complete two-turn agentic exchange: the user asks a question requiring a tool, the model emits a tool call, the engine executes it, and the model produces a final answer incorporating the result.

```
<!-- Turn 1: User prompt, formatted by ChatTemplate -->

<|im_start|>system
You are a helpful assistant with access to tools.

Available tools:
{"type":"function","function":{"name":"calculator",
 "description":"Evaluate a mathematical expression",
 "parameters":{"type":"object","properties":{
   "expression":{"type":"string","description":"Math expression to evaluate"}},
   "required":["expression"]}}}
<|im_end|>
<|im_start|>user
What is the square root of 17161?<|im_end|>
<|im_start|>assistant


<!-- Turn 1: Model generates a tool call -->

<tool_call>
{"name": "calculator", "arguments": {"expression": "sqrt(17161)"}}
</tool_call>
<|im_end|>


<!-- Turn 2: ToolEngine appends tool result and re-prompts -->

<|im_start|>tool
{"name": "calculator", "result": "131.0"}
<|im_end|>
<|im_start|>assistant


<!-- Turn 2: Model produces final answer (no tool call) -->

The square root of 17,161 is exactly 131.
<|im_end|>
```

The `ToolParser` detects the `<tool_call>` block in Turn 1's output, parses `name` and `arguments`, and dispatches to the registered `calculator` handler. The result string `"131.0"` is formatted by `ChatTemplate::append_tool_result()` as the `tool` role message, and the loop re-enters `InferenceEngine::generate()` for Turn 2. Since Turn 2 contains no `<tool_call>` block, the agentic loop terminates and the final text is returned to the caller.